

3주차 정리

자습과제 1.

디지털대성 " 두 테이블 삭제 문제" 트랜잭션 관점에서 다시 분석

1. 발생이유 : 둘 다 프라이머리 키가 걸려있었고, 하나가 삭제되고 나서 다른 하나도 반드시 삭제가 되어 하는 로직으로 구성해야 하는데, 테스트 당시에는 둘다 계속 지워져서 아무 문제가 없었는데 이제 실서버에 적용하고 나서 확인해보니 페이지 오류나 어떤 상황으로 인해 한쪽 만 지워지고 다른 한쪽은 안 지워지는 상황이 발생했습니다.

2. 그 상황에서 @Transactional 적용

예시)

@Transactional

```
public void deleteData(Long studentId) {  
    // 1. 분/초 테이블 데이터가 먼저 삭제 (자식테이블부터)  
    timeRepository.deleteByStudentId(studentId);  
  
    // 2. 만약 여기서 예러가 난다면 대처하는 방법  
    If (checkSomething()) {  
        Throw new RuntimeException("예기치 못한 상황 발생!");  
    }  
  
    // 3. 날짜 테이블(부모) 데이터 삭제  
    dateRepository.deleteByStudentId(studentId);  
}
```

일단 날짜와 분/초 테이블로 나뉘어진 상태였고 키 값이 같았기 때문에 (여기서 키 값은 학생 고유 번호를 의미함) 롤백을 포함하는 @transcational 을 사용하여 한쪽 테이블이 삭제가 되더라도 문제가 생기면 롤백을 하게 하여서 각각 사라지는 문제를 없앨 수 있습니다.

비관적 락과 낙관적 락 정리:

비관적 락은 충돌이 자주 일어날 경우를 대비해 미리 잠가두는 것을 애기하고 잔액 차감, 재고 차감 등 충돌이 자주 발생하고 정확성이 매우 중요한 경우에 사용합니다.

낙관적 락은 충돌이 거의 안 일어나고 예외를 던지고 종료해 버립니다.

낙관적 락 예시)

```
UPDATE TABLE_NAME SET name = "수정완료", version = 2 -- 버전을 1 올림 WHERE id = 1 AND  
version = 1 -- "내가 읽었을 때 버전이 1 인 경우만" 수정해라!
```

다른 사람이 먼저 수정을 했다면 버전이 올라가 있을 것이고 where version = 1 조건에 맞는 데이터가 없으니 0 건 수정이 되고 스프링은 누가 수정을 했다고 에러를 알려줍니다.

자습과제 2. 동시성 실습 직접 돌려보기

1. RaceConditionDemo

```
<terminated> RaceConditionDemo [Java Application] C:\Program Files\Ecl
```

```
기대값 : 1000000  
unsafe (race condition): 985155  
synchronized: 1000000  
AtomicInteger: 1000000
```

```
Problems @ Javadoc Declaration Console X  
<terminated> RaceConditionDemo [Java Application] C:\Program Files\Eclipse Adoptium\jdk-17.0.18.8-hotsp  
기대값 : 1000000  
unsafe (race condition): 985166  
synchronized: 1000000  
AtomicInteger: 1000000
```

Unsafe 결과가 다르게 나오고 있습니다.

2. MapConcurrencyDemo

```
Problems @ Javadoc Declaration Console X  
<terminated> MapConcurrencyDemo [Java Application] C:\Program Files\Eclipse Adoptium\jdk-17.0.18.8-hotsp  
HashMap: 기대 size 10000, 실제 size 6927  
ConcurrentHashMap: 기대 size 10000, 실제 size 10000
```

선택과제 1

(선택) Virtual Threads 실습 — Java 21+ 환경 VirtualThreadsDemo 를 실행해서 플랫폼 스레드와 가상 스레드의 처리 시간 차이 확인. 환경이 안 되면 결과만 보고 넘어가도 OK.

결과값

플랫폼 스레드: 약 50,000ms(50초) 이상 소요됩니다. (200개씩 50번 나누어 처리하기 때문)

가상 스레드: 약 1,000ms ~1,500ms 소요됩니다. (10,000개를 한꺼번에 처리하기 때문)